Variables, Data Types & Basic I/O

Suman 19 Jan, 2026 At 6:00 PM Pre-Reads Trimester-1 Recommended Resource

Associated Lectures (1)

Description Discussions

SESSION 1.2 PRE-READ

Type Conversion, Operators, Conditionals & Strings

Program: AIM101 — Artificial Intelligence & Machine Learning Foundation **Pre-read Status:** MANDATORY — Must be completed before live session

Session 1.1 Recap: What You Already Know

Before Session 1.2, make sure you're comfortable with these concepts from Session 1.1:

Variables

Variables are named containers that store data:

```
name = "Priya"    # str - text
age = 25          # int - whole number
height = 5.6      # float - decimal
is_student = True # bool - True/False
```

Data Types

Python has four fundamental data types:

Type	Example	Purpose
`int`	`42`	Whole numbers
`float`	`3.14`	Decimal numbers

Type	Example	Purpose
`str`	`"Hello"`	Text (in quotes)
`bool`	`True` / `False`	Yes/No values

The type() Function

Use `type()` to check what type a value is:

```
print(type(42))      # <class 'int'>
print(type("Hello")) # <class 'str'>
```

The Critical Type Mismatch Problem

This was the key lesson — types matter!

```
5 + 3     # = 8    (math)
"5" + "3" # = "53" (text concatenation)
"5" + 3   # = ERROR! (can't mix)
```

What You Will Learn Today

By the end of Session 1.2, you will be able to:

Convert between data types using `int()`, `float()`, `str()`, `bool()`
Apply all arithmetic operators including `//`, `%`, and `**`
Manipulate strings with indexing, slicing, and methods
Format output professionally using f-strings
Build interactive programs with `print()` and `input()`
Compare values using comparison operators (`==`, `<`, `>`, etc.)
Combine conditions using logical operators (`and`, `or`, `not`)
Make decisions with `if` / `elif` / `else` statements
Create nested conditionals for complex decision trees
Complete a mini-project: Mars Redemption Unit Converter

The Story Continues: Mars Redemption

Remember the Mars Climate Orbiter?

In Session 1.1, we learned about the $327.6 million disaster caused by a unit mismatch. Lockheed Martin sent values in pound-force-seconds while NASA expected Newton-seconds.

Today's Mission

You will build the unit converter that could have saved Mars!

By the end of this session, you'll have a working program that:

- Displays a professional menu
- Gets user input
- Validates the input
- Performs the correct conversion
- Displays formatted results

This requires EVERYTHING we'll learn today — type conversion, operators, strings, f-strings, I/O, and conditionals.

Key Vocabulary

Before class, familiarize yourself with these terms:

Type Conversion

Term	Meaning
Type Conversion	Changing data from one type to another
Truncation	Cutting off decimal places (not rounding!)
Truthy/Falsy	Values that convert to True or False

Operators

Term	Meaning
Floor Division (`//`)	Division that rounds DOWN to nearest integer
Modulo (`%`)	Remainder after division
Exponentiation (`**`)	Raising to a power

Strings

Term	Meaning
Indexing	Accessing a single character by position
Slicing	Extracting a portion of a string
F-string	Formatted string literal (starts with `f`)

Conditionals

Term	Meaning
Comparison	Asking if two values are equal, greater, less, etc.
Logical	Combining conditions with `and`, `or`, `not`
Conditional	Code that runs only if a condition is True
Nested	A conditional inside another conditional

Setup Reminder

Quick Environment Check (60 seconds)

Your environment should already be set up from Session 1.1. Just verify:

Open Terminal/Command Prompt

Activate your environment:

```
conda activate mlenv
```

Check Python version:

```
python --version
```

Expected: `Python 3.12.x`

Launch Jupyter:

```
jupyter notebook
```

Expected: Browser opens with Jupyter interface

If any step fails, refer back to the Session 1.1 Pre-read for troubleshooting.

Things to Ponder

Think about these questions before class. We'll explore them during the session.

Type Conversion Questions

Why would `int("3.14")` fail, but `int(3.14)` succeed?

- What's the difference between a string and a float?
- How would you convert "3.14" to an integer?

What does `bool("False")` return and why?

- Remember: strings and boolean values are different!
- What makes a string "truthy"?

Operator Questions

What is `17 % 5` and when would you use modulo?

- Think about: checking if a number is even/odd
- Think about: wrapping around (like clock hours)

Why does `-7 // 2` equal `-4` instead of `-3`?

- Hint: Floor division rounds toward negative infinity
- This is different from truncation!

String Questions

If `text = "Python"`, why does `text[1:4]` give `"yth"` not `"ytho"`?

- What's special about the stop index?
- Why did Python designers make this choice?

Conditional Questions

Why do we need `elif` when we could just use multiple `if` statements?

- What's different about how they execute?
- When does it matter?

What's the difference between `=` and `==`?

- This is the #1 beginner mistake!
- What happens if you confuse them?

Preview: The Mars Redemption Unit Converter

Here's a sneak peek of what you'll build by the end of class:

```
=======================================
     MARS REDEMPTION UNIT CONVERTER
=======================================

Choose conversion type:
1. Pounds (lbf-s) → Newtons (N-s)
2. Kilograms → Pounds
```

```
3. Celsius → Fahrenheit
4. Miles → Kilometers

Enter choice (1-4): 1
Enter value to convert: 4.45

Converting 4.45 lbf-s to N-s...
Result: 19.79 N-s

✓ Mars is safe this time!
========================================
```

This program uses:

- **Type Conversion**: `float(input(...))` to get numbers
- **Arithmetic Operators**: `*` for conversion calculations
- **Strings**: Menu display
- **F-Strings**: `f"{result:.2f}"` for formatted output
- **I/O**: `print()` and `input()` for user interaction
- **Comparison**: Validating menu choice
- **if/elif/else**: Selecting which conversion to perform

Historical Context: More Unit Conversion Disasters

The Mars Orbiter isn't the only unit-related disaster. Here are a few more that show why type safety and unit conversion matter:

Air Canada Flight 143 (1983)

- **What happened**: Plane ran out of fuel mid-flight
- **Cause**: Ground crew calculated fuel in pounds, but the new plane used kilograms
- **Result**: Emergency landing on an abandoned airstrip (everyone survived)
- **Conversion factor missed**: 1 kg = 2.205 lbs

Vanguard Climate Satellite (1999)

- **What happened**: Satellite separated from rocket at wrong altitude
- **Cause**: Programmers confused English and metric units in software
- **Result**: $154 million satellite destroyed
- **Same year as Mars Orbiter!**

Korean Air Flight 007 (1983)

- **What happened**: Plane strayed into Soviet airspace
- **Cause**: Crew entered magnetic heading instead of true heading
- **Result**: Shot down by Soviet interceptors
- **Unit confusion**: Magnetic vs. True North (declination)

The lesson: Even small type mismatches can have catastrophic consequences!

Explore Further (Optional)

Python Official Documentation

- **Numbers**: https://docs.python.org/3.12/tutorial/introduction.html#numbers
- **Strings**: https://docs.python.org/3.12/tutorial/introduction.html#strings
- **Control Flow**: https://docs.python.org/3.12/tutorial/controlflow.html

Interactive Practice

- **Type Conversion Practice**: Try these in your Jupyter notebook:

```
int("42")
float("3.14")
str(42)
bool(0)
bool("hello")
```

See You in Class!

Vishlesan i-Hub IIT Patna × Masai School